A

Project Report

on

Predictive Maintenance of Aircraft Engines

In partial Fulfillment of the Requirements

for the Award of the Degree of

BACHELOR OF ENGINEERING

IN

INFORMATION TECHNOLOGY

Submitted by

Sumedhaa Medavarapu	160118737019
Usha Gourigari	160118737021

SUPERVISOR

Mr.S.Rakesh

Asst.Professor

DEPARTMENT OF INFORMATION TECHNOLOGY

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY(A)

(Affiliated to Osmania University;Accredited by NBA,NAAC,ISO)

Kokapet(V),Gandipet(M),Hyderabad-500075

Website:www.cbit.ac.in

2021-2022

Declaration

We declare that the project work **Predictive Maintenance of Aircraft Engines** submitted to the department of Information Technology, Chaitanya Bharathi Institute of Technology, Hyderabad in fulfillment of degree for the award of Bachelor of Engineering is a bonafide work done by us which was carried under the supervision of **Mr.S.Rakesh**.

Also, we declare that the matter embedded in this report has not been submitted by us in full or partial thereof for the award of any degree/diploma of any other institution or university previously.

No part of the work is copied from books/journals/internet and wherever the portion is taken, the same has been duly referred in the text. The report is based on the project work done entirely by us and not copied from any other source.

Sumedhaa Medavarapu

Usha Gourigari

CERTIFICATE

This is to certify that the project work entitled **Predictive Maintenance of Aircraft Engines** is submitted by **Sumedhaa Medavarapu (160118737019)** & **Usha Gourigari(160118737021)** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Engineering** in **Information Technology** to **CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY(A)** affiliated to **OSMANIA UNIVERSITY**,Hyderabad is a record of bonafide work carried out by them under my supervision and guidance.The results embodied in this report have not been submitted to any other University or Institute for the award of any other Degree or Diploma.

Signature of the Supervisor
Mr.S.Rakesh
Assistant Professor

Signature of the HOD
Dr.K.Radhika
Professor and Head, IT

Acknowledgement

The satisfaction that accompanies the successful completion of the task would be put incomplete without the mention of the people who made it possible, whose constant guidance and encouragement crown all the efforts with success.

We wish to express our deep sense of gratitude to **.S.Rakesh**, Designation and Project Supervisor, Department of Information Technology, Chaitanya Bharathi Institute of Technology, for his able guidance and useful suggestions, which helped us in completing the project in time.

We are particularly thankful to **.K.Radhika**, the Head of the Department, Department of Information Technology, his guidance, intense support and encouragement, which helped us to mould our project into a successful one.

We show gratitude to our honorable Principal **Dr.P.Ravinder Reddy**, for providing all facilities and support.

We avail this opportunity to express our deep sense of gratitude and heartful thanks to **Mr.Subash Garu**, President, CBIT for providing a congenial atmosphere to complete this project successfully.

We also thank all the staff members of Information Technology department for their valuable support and generous advice. Finally thanks to all our friends and family members for their continuous support and enthusiastic help.

Sumedhaa Medavarapu 160118737019

Usha Gourigari 160118737021

Abstract

In the past if a mechanic was servicing a machine and found unusual visual or acoustic behaviour in a certain part, the machine may be shut down before breaking and the part was exchanged.That is already predictive maintenance. The new thing today is that machines and algorithms can perform this task 24/7 based on smart algorithms, more data and automated alerts for detecting irregularities.In predictive maintenance scenarios, data is collected over time to monitor the state of equipment.As in most ML projects, there is a need for enough historical data to understand previous failures.

Applications of predictive maintenance are massive cost savings mainly due to decreased downtime,avoidance of related machinery failure due to connected parts,reduction of storage cost for spare parts.Predictive maintenance determines the condition of equipment and predicts when maintenance should be performed.The goal is to find patterns that can help to predict and ultimately prevent failures.

Failure prediction is a major topic in predictive maintenance in many industries. Aircrafts are particularly interested in predicting equipment failures in advance so that they can enhance their operations and reduce delays.The aim of predictive maintenance in aviation is first to predict when a component failure might occur, and secondly, to prevent the occurrence of the failure by performing maintenance..Observing engine's health and condition through sensors and telemetry data is assumed to facilitate this type of maintenance by using machine learning algorithms.

Keywords: predictive maintenance,machine learning algorithms.

Table of Contents

List of Tables

List of Figures

Abbreviations

Abbreviation	Description
ML	Machine learning
CNN	Convolutional neural network
LSTM	Long Short-Term Memory
ReLU	Rectified linear activation unit
CLSTM	Convolutional Long Short-Term Memory
XGBoost	Extreme Gradient Boosting
LightGBM	Light Gradient Boosting Machine
RMSE	Root mean square erro
MSE	mean square erro
RUL	Remaining useful life
TTF	Time To Failure
RNN	Recurrent Neural Network

CHAPTER 1

Introduction

1.1 Overview

In the past if a mechanic was servicing a machine and found unusual visual or acoustic behaviour in a certain part, the machine may be shut down before breaking and the part was exchanged. That is already predictive maintenance. The new thing today is that machines and algorithms can perform this task 24/7 based on smart algorithms, more data and automated alerts for detecting irregularities.

Predictive maintenance is challenging to implement but the potential benefits outperform the efforts by far. While there are various reasons why an actual deployment in many cases takes more time and effort than expected.In predictive maintenance scenarios, data is collected over time to monitor the state of equipment.As in most ML projects, there is a need for enough historical data to understand previous failures.

Moreover, general static features of the system can also provide valuable information, such as mechanical properties, average usage and operating conditions.The goal is to find patterns that can help to predict and ultimately prevent failures.

1.2 Applications

- Massive cost savings mainly due to decreased downtime.
- Avoidance of related machinery failure due to connected parts.
- Reduction of storage cost for spare parts.
- The maintenance team can focus on long term improvements rather than emergency repairs.

- A precious treasure of data will be generated that can be used for all kinds of other optimising tasks.

1.3 Problem Definition

Failure prediction is a major topic in predictive maintenance in many industries. Aircrafts are particularly interested in predicting equipment failures in advance so that they can enhance their operations and reduce delays.Observing the engine's health and condition through sensors and telemetry data is assumed to facilitate this type of maintenance by using machine learning algorithms.

1.4 Objective

Predictive maintenance determines the condition of equipment and predicts when maintenance should be performed.In predictive maintenance scenarios, data is collected over time to monitor the state of equipment. The goal is to find patterns that can help to predict and ultimately prevent failures.The aim of predictive maintenance in aviation is first to predict when a component failure might occur, and secondly, to prevent the occurrence of the failure by performing maintenance.

CHAPTER 2

Literature Survey

2.1 Paper 1

2.1.1 Title:Predictive Maintenance of Aircraft Engine using Deep Learning Technique.

2.1.2 Authors :Ade Pitra Hermawan, Dong-Seong Kim and Jae-Min Lee.

2.1.3 Site and Year of Publication: IEEE,2020.

2.1.4 Description: In this paper, an accurate algorithm to estimate remaining useful life of aircraft engine is proposed. Since the aircraft engine has a low fault tolerant, meaning that a little faulty in the system can lead to catastrophic conditions, an accurate and real-time information about the engine condition is required. This paper utilizes the combination of CNN and LSTM algorithms in learning the behavior of the historical data and providing the accurate information about the time to failure of the system. The simulation results demonstrate that the proposed system is able to achieve improved performance in terms of accuracy rate and computing time compared to the previous works.

The detailed contributions of this paper are as follow:

• To provide more accurate information, a combination of CNN and LSTM algorithm is proposed in this paper, this combination aims to learn the feature of the data completely.

• Instead of providing one information to the user, we designed multi step faulty information that shows when the faulty will be happened in one period of time.

• Instead of using a big number of hidden neurons, we apply a small number of neurons during training and testing to achieve low computation.

• To show the robustness of the proposed algorithm, we compare the performance of the proposed LSTM with different architectures and different algorithms.

Here is a figure attached :

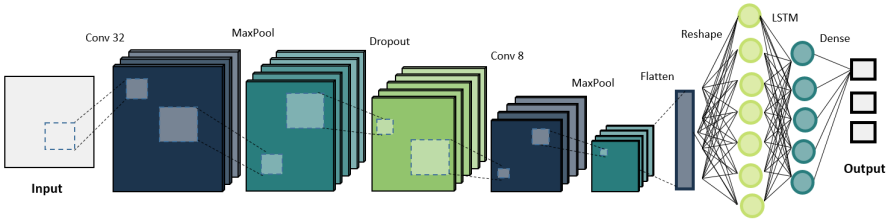

Figure 2.1: Architecture of the proposed system in paper 1.

The paper proposed the combination of CNN and LSTM algorithm, further will be called as CLSTM algorithm. The main idea of the proposed architecture is the combination of CNN and LSTM algorithm. The reason of this combination is because the LSTM algorithm is proven good at handling time series data, while CNN is able to extract the features better than the other algorithms. The detailed of the proposed architecture can be seen in Fig. 2.1.

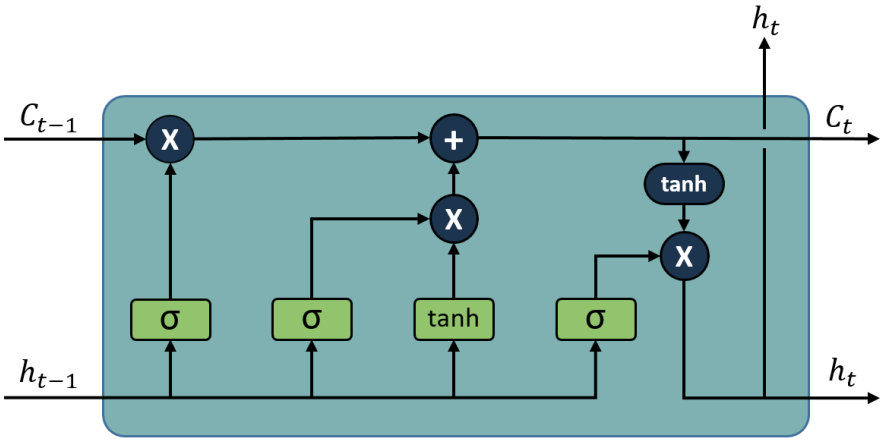

Figure 2.2: LSTM network of paper 1.

In the proposed architecture, convolutional layer was used for the first layer to process the input data. After that max pooling was used with kernel size of 1. Dropout layer with value 0.1 also used after the max pooling layer to avoid over- fitting in the system. Rectified linear unit (ReLU) activation function is used in this paper, which transforms the output into 0 if the input value is less than 0, and transforms to X if the input value more than 0, as shown in the below equation: Lastly, the dense layer with three of hidden layers is

used associated with the number of class. On the last dense layer, we used a softmax activation function to interpret the output optimally 1) LSTM: The LSTM layer that we used in this system consists of 5 hidden layers. Fig. 2.1 shows the illustration of hidden neuron in LSTM. As it can be seen from Fig. 2.1, that in LSTM networks utilized two different inputs to produce the output. These inputs defined as a ct and ht respectively which are coming from the previous layers. The detail information regarding to internal gates inside LSTM algorithm.

Simulation results to evaluate the effectiveness of the proposed system is presented. The simulations were conducted using Keras library on top of Google Colaboratory with GPU Geforce RTX 2070, 24GB DDR4 RAM. All algo- rithms were simulated using the same parameters, with one time training, and in the same environment. The paper simulated CNN, LSTM, and CLSTM algorithms and compared the performance of each algorithm in terms of accuracy rate. Based on the simulations, the highest accuracy rate achieved by the utilization of CLSTM algorithm. As it can be seen from Table 2.1.

Here is a table attached :

Table 2.1: Performance comparison on accuracy for paper 1

$Metrics$	$AccuracyRate$
CNN	89.33
LSTM	97.32
Proposed CLSTM	99.60

In this paper, a combination of CNN and LSTM algorithm to estimate the remaining useful life of aircraft engine is pro- posed. The CLSTM algorithm that we proposed successfully learn the historical data completely, and provide accurate infor- mation regarding to the engine's condition. The convolutional layer helps the system to learn the features of historical data, where LSTM layer is able to memorize past information that used for estimating the future data. Based on the simulation results, it is showed in the paper that the CLSTM achieved the best performance in terms of accuracy rate compared

to the other algorithms. Furthermore, energy efficient based approach will be discussed in the future of this study.

2.2 Paper 2

2.2.1 Title : Fault Knowledge Transfer Assisted Ensemble Method for Remaining Useful Life Prediction

2.2.2 Author:Pengcheng Xia ,Yixiang Huang,Peng Li, Chengliang Liu, and Lun Shi.

2.2.3 Site and Year of Publication:IEEE Transactions On Industrial Informatics,2022.

2.2.4 Description: In this paper,Data-driven methods have been widely studied and applied, however, almost all the researches learn degradation trends regardless of different fault conditions, which can lead to different degradation patterns.This paper proposes a novel fault information assisted RUL prediction method based on a convolutional long short-term memory (LSTM) ensemble network, where fault conditions are obtained via fault knowledge transfer. Divergence minimization and domain adversarial adaptation are combined to transfer fault knowledge from a fault dataset to the run-to-failure data in a weakly supervised manner.With the predicted fault information,the RUL prediction network can learn various degradation patterns under different faults separately using a structure of multiple LSTMs. Then an ensemble strategy based on soft fault conditions is designed to get final RUL prediction results.

In paper,a fault knowledge transfer assisted RUL prediction method is proposed. The proposed method consists of two important parts: one is the FTNN, and the other is the convolutional LSTM ensemble network for RUL prediction.The proposed method aims to utilize abundant fault knowledge in the existing fault datasets to assist the RUL prediction task to get prior fault information. Since the fault datasets and the RUL prediction datasets usually have different data distributions, it is improper to directly use the fault knowledge across dataset. After the fault condition of each sample is predicted by the FTNN, a convolutional LSTM ensemble network, which uses the fault information as prior information is proposed to get more accurate

RULs.

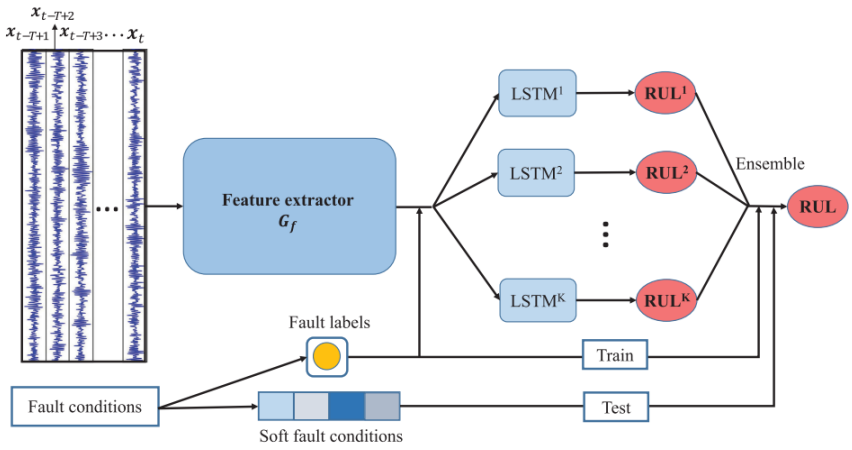

Figure 2.3: Architecture of the proposed system in paper 2.

The bearing run-to failure data used for RUL prediction are from accelerated life tests on PRONOSTIA platform.The first convolutional layer of the feature extractor uses wide kernels to suppress high frequency noise just like the WDCNN.The paper set the kernel size of the first layer as 64 and stride as 16.The proposed method aims to utilize abundant fault knowl- edge in the existing fault datasets to assist the RUL prediction task to get prior fault information. Since the fault datasets and the RUL prediction datasets usually have different data distribu- tions, it is improper to directly use the fault knowledge across dataset. Transfer learning is an effective and unique method to address this problem. Therefore, we use transfer learning, or more specifically, domain adaptation technique to develop a fault knowledge transfer network. Since knowledge transfer across datasets in this manuscript can be challenging, we combine divergence minimization and domain adversarial adaptation to ensure the knowledge transfer ability. Domain adaptation must base on a feature extractor used to learn domain-invariant fea- tures. In the proposed method, the input of network is raw signals with noise. CNN is the most suitable and powerful network to extract features from raw signals according to many previous literatures in diagnosis and prognosis fields. Therefore, CNN and transfer learning combining adversarial mechanism are integrated to construct the FTNN.

Module	No.	Layer	Kernel/Hidden size	Stride	Kernel number	Activation	Output size
Feature Extractor	1	Conv1	64	16	32	ReLU	72×32
	2	Pool1	2	2	32	/	36×32
	3	Conv2	5	1	64	ReLU	32×64
	4	Pool2	2	2	64	/	16×64
	5	Conv3	5	1	64	ReLU	12×64
	6	Pool3	2	2	64	/	6×64
Condition Classifier	C7	FC	100	/	1	ReLU	100×1
	C8	FC	32	/	1	ReLU	32×1
	C9	Output	4	/	1	Softmax	4×1
Domain Classifier	D7	FC	32	/	1	ReLU	32×1
	D8	Output	2	/	1	Softmax	2×1
LSTM Module	L7	LSTM	64	/	/	/	64×1
	L8	Dropout	/	/	/	/	64×1
	L9	Output	1	/	/	/	1

Figure 2.4: Details of the proposed system in paper 2.

The fault condition of each sample is predicted by the FTNN, a convolutional LSTM ensemble network, which uses the fault information as prior information is proposed to get more accurate RULs. In the proposed RUL prediction network, we propose to use multiple subnetworks to model degradation patterns under different fault modes, respectively. Since degra- dation patterns rely on temporal information, whereas CNN is not good at temporal modeling, LSTM, which has strong temporal modeling ability is used to learn degradation patterns from the feature representations extracted by CNN.

In this paper, they have proposed a novel fault knowledge transfer assisted convolutional LSTM ensemble method for RUL predic- tion. Divergence minimization and domain adversarial adapta- tion techniques were combined to transfer fault knowledge from a fault dataset to the run-to-failure samples. With the diagnosed fault condition information, the RUL prediction network can learn various degradation patterns under different faults using the structure of multiple LSTMs. Then, an ensemble process based on predicted soft fault conditions were proposed to get RUL prediction results. Experiments on bearing datasets vali- dates the effectiveness and superiority of our proposed method. For further researches and applications, this proposed method presents a new prognostic paradigm by diagnosing fault condi- tions to improve the RUL prediction results, which can also pro- vide some guidance for researches on the degradation patterns under different fault types. This may contribute to

understand- ing the machinery degradation mechanisms and constructing some hybrid models [39]. Because of the more accurate RUL prediction performance, our model can be integrated into some complex industrial prognosis systems. For example, the RUL prediction algorithm can be integrated into the key-performance- indicators oriented prognosis systems [40] to improve the production system reliability or integrated into some industrial cyberphysical

2.3 Paper 3

2.3.1 Title: Aircraft Engine Reliability Analysis using Machine Learning Algorithms

2.3.2 Author: Deepankar Singh, Mithilesh Kumar, K.V. Arya and Sunil Kumar

2.2.3 Site and Year of Publication: IEEE,2020

2.3.4 Description: In the aviation industry, the reliability analysis of aircraft engines is essential for ensuring the smooth functioning of each component of an aircraft engine. The reliability analysis is also important to predict their scheduled maintenance event and the Remaining Useful Life (RUL) of engine parts. Existing approaches for engine reliability are based on numerical methods, which do not predict RUL accurately. Hence, a more accurate model is required for predicting maintenance events. The reliability of an aircraft engine can be measured using readings of different sensors. In this paper, the performances of different machine learning algorithms are studied, and finally, a better algorithm is suggested for predicting RUL. Additionally, a classification approach is proposed to classify the health state of an engine. The experimental results show that the XGBoost gives the best prediction accuracy in terms of root mean square error. The proposed LightGBM-based classifier further enhances the maintenance prediction based on the health state of the aircraft engine. Thus, the proposed analysis shows that XGBoost and LightGBM is a better choice for predicting the RUL, and for classifying the health state of the aircraft engine.

A.Dataset collection for RUL analysis The dataset used in this paper is the National Aeronautics and Space Administration (NASA) data repository

for aircraft engine reliability. The dataset gives the practical failure time of different sensors of different turbofan engines. Almost all the turbofan engines are of the same type but each of the engines begins with a different initial condition (wear and tear) which are unknown to the end-user. The working of each of the engines can be altered by altering the three operational settings. Each engine is assigned a total of 21 sensors to collect data regarding different conditions during its runtime. In the beginning, each engine starts with a well functioning condition. After a few cycles of runtime, each engine develops some error and gradually comes to a halt. A total of three datasets were given to prepare, test, and gauge the exactness of a component. The available dataset is referred with training information, testing information, and the corresponding ground truth information.

B.Schematic work-flow of the proposed model

Starting from collecting the sensor data and preprocessing it before working to the training and testing of the dataset using different machine learning models and comparing their classification and regression efficiency. Finally the paper evaluated the model's performances and showing the results in the form of graphs.

C.Study of different machine learning algorithms on engine reliability A python-based programming environment is used to test the dataset for different models. Firstly, the dataset text file was read, and then, using python language, regression, and classification algorithms were applied to it. The positive side of using the Python environment is that the method of processing is simple and the speed of processing is high, which helps to easily simulate the results. The stored results were then plotted for better visualization.

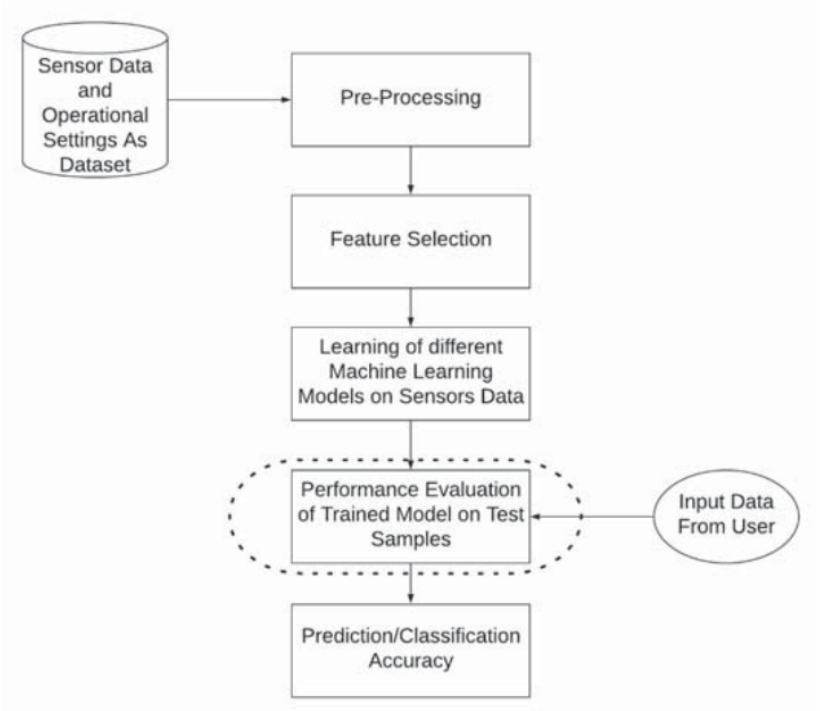

Figure 2.5: WorkFlow diagram of paper 3.

In this paper algorithm such as Random forest classifier,LightGBM,Ensemble method,XGBoost have been selected for classifying state of the engine into different classes like high, medium, and low.

Table 2.2: Performance Comparision on Accuracy for paper 3

Metrics	*AccuracyRate*
Naive Bayes	88.62
Catboost	95.02
XGBOOST	96.00
Random Forest	95.28
Ensemble method	0.95

Table 2.3: Performance Comparision on RMSE for paper 3

Metrics	*RMSE*
Ada-boost	46.49
Decision tree	44.16
XGBOOST	24.24
Random Forest	33.49
Support vector machine	30.67

The main purpose of this paper is to find the best algorithm for machine learning to predict the remaining useful life of a part of an aircraft engine. Additionally, this paper also finds a better classifier which can classify the health state of an engine into three different labels i.e., high, medium, and low. The classification performances of different algorithms are shown in Table II. It can be observed that LightGBM gives the highest accuracy compared to other selected algorithms. LightGBM is a decision tree-based classifier. Our primary concern was to reduce the error between the remaining useful life and the estimated remaining useful life in the prediction problem. The paper compared the test results with the actual estimates of the RUL available in the dataset for the underlined data collection. The root mean squares error values have been plotted, and the XGBoost algorithm will show that the best result is obtained.

2.4 Paper 4

2.4.1 Title: Prediction of Remaining Useful Lifetime (RUL) of Turbofan Engine using Machine Learning

2.4.2 Author: Vimala Mathew, Tom Toby, Vikram Singh, B Maheswara Rao, M Goutham Kumar

2.4.3 Site and Year of Publication: IEEE,2017

2.4.4 Description: Maintenance of equipment is a critical activity for any business involving machines. Predictive maintenance is the method of scheduling maintenance based on the prediction about the failure time of any equipment. The prediction can be done by analyzing the data measurements

from the equipment. Machine learning is a technology by which the outcomes can be predicted based on a model prepared by training it on past input data and its output behavior. The model developed can be used to predict machine failure before it actually happens.

There are different approaches available for developing a machine learning model. In this paper, a comparative study of existing set of machine learning algorithms to predict the Remaining Useful Lifetime of aircraft's turbo fan engine is done. The machine learning models were constructed based on the datasets from turbo fan engine data from the Prognostics Data Repository of NASA. Using a training set, a model was constructed and was verified with a test data set. The results obtained were compared with the actual results to calculate the accuracy and the algorithm that results in maximum accuracy is identified. We have selected ten machine learning algorithms for comparing the prediction accuracy.

Machine Learning (ML) enables computers to do selflearning from the data available, without any explicit programs. These machines can react to new data, based on its past learning. Prediction is one of the important applications of Machine Learning. There are two categories of ML. They are categorized as Supervised and Unsupervised [13] In supervised learning, the dataset includes their output classes which are used to train the machine whereas in unsupervised learning output class information is not available; instead the data is partitioned into unknown classes. The methodology adopted in this paper is to do prediction based on supervised machine learning. The turbo fan engine dataset includes a training set and a test set. Both training and test data includes the actual RUL value which we plan to predict. The machine is trained using the training set and is tested for accuracy using the test data.The different algorithms were compared to obtain the prediction model having the closest prediction of remaining useful lifecycle in terms of number of life cycles.

The dataset is taken from NASA data repository. The dataset selected includes the run-to-failure sensor measurements from degrading turbofan engines. Although the turbo fan engines are of same type, each engine starts

with different degree of initial conditions and there are variations in the manufacturing process of the engines, which are not known to the user. For the turbo fan engines under consideration, the performance of each engine can be changed by adjusting three operational settings. Each engine has 21 sensors collecting different measurements related to the engine state at runtime. The main characteristic of the dataset is that it is a time series data.

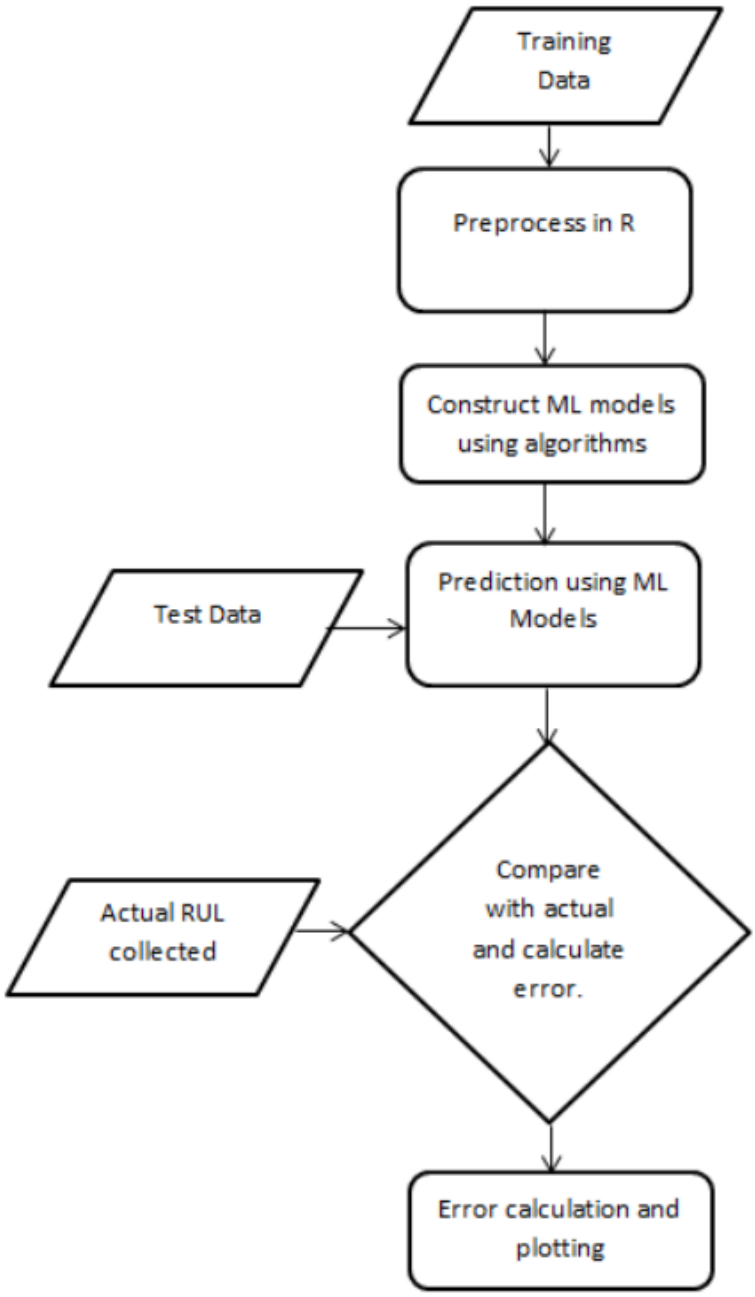

Figure 2.6: WorkFlow diagram of paper 4.

Training and evaluation of the Machine Learning Model was carried out using four data sets, each containing data collected from 100-250 engines and each engine having 21 different sensor values.All ten algorithms were tested using all four datasets.If actual and predicted points coincide, it indicates maximum accuracy. In the second set, the variation of actual RULs from the predicted of dataset1 are plotted in the form of a regression line.For the most accurate algorithm, maximum points will be on the regression line. In the third set, the Root Mean Square Error (RMSE) values between actual RUL and predicted RUL were plotted.The RMSE of individual algorithms were compared for consistency between different data sets, by preparing a Root Mean Square Error Table. The objective was to obtain the algorithm having lowest RMSE Value. Even though cent percent accuracy cannot be achieved, error in prediction can be minimized by selecting the best algorithm. On observation it was found that random forest algorithm generates the least error.

Algorithm	*Data Set 1*	*Data Set 2*	*Data Set 3*	*Data Set 4*	*Mean*
Linear Regression	29.91	31.49	45.64	39.81	36.71
Decision Tree	28.48	34.52	27.74	45.91	34.17
SVM	48.17	31.12	61.53	34.65	43.86
Random Forest	24.95	29.64	30.55	33.79	29.73
KNN	30.79	34.79	34.44	44.70	36.18
K Means	78.30	90.19	72.92	95.45	84.21
Gradient Boost	27.45	33.35	31.78	39.30	32.97
Ada boost	28.82	33.84	30.91	39.16	33.18
Deep Learning	29.62	42.41	46.82	38.11	39.24
Anova	33.50	41.14	35.46	51.44	40.38

Figure 2.7: Comparison of different algorithms in paper 4.

The different algorithms were evaluated in the same way on the same data for consistent comparison of results. While predicting RUL, the main objective is to reduce the error between the actual RUL and the predicted RUL. For each dataset, the test results were compared with the actual values of the RUL available in the data set. The root mean squares of the errors were plotted and it is observed that the best results were obtained by random forest algorithm. Random forest algorithm captures the variance of several input variables at the same time and enables high number of observations to take part in the

prediction. It was observed that the performance of all ten algorithms were consistent in the four different datasets, generating proportional accuracy for the different algorithms tested.

2.5 Paper 5

2.5.1 Title: Predictive Maintenance in Aviation: Failure Prediction from Post Flight Reports

2.5.2 Author: Panagiotis Korvesis, Stephane Besseau, Michalis Vazirgiannis

2.5.3 Site and Year of Publication: IEEE,2018

2.5.4 Description:

In this paper the authors present an approach to tackle the problem of event prediction for the purpose of performing predictive maintenance in aviation. Given a collection of recorded events that correspond to equipment failures, our method predicts the next occurrence of one or more events of interest (target events or critical failures). The objective is to develop an alerting system that would notify aviation engineers well in advance for upcoming aircraft failures, providing enough time to prepare the corresponding maintenance actions. The authors formulate a regression problem in order to approximate the risk of occurrence of a target event, given the past occurrences of other events. In order to achieve the best results we employed a multiple instance learning scheme (multiple instance regression) along with extensive data preprocessing. They applied the method on data coming from a fleet of aircraft and our predictions involve failures of components onboard, specifically components that are related to the landing gear. The event logs correspond to post flight reports retrieved from multiple aircraft during several years of operation.

Monitoring systems generate events that reflect the condition of the equipment or the processes that are being observed. Hardware health, business processes and server activities are examples of cases where event logs are generated in order to provide the corresponding information. Given a set of event logs, questions of great importance are: how can we predict when a specific event will occur? How far before its occurrence are we able to predict

it? In this paper we address the problem of predicting the next occurrence of a specific event (target event), given the history of past events. This problem is extremely significant in predictive maintenance.

sys id	timestamp	event id	source	description
sys_1	2013-11-10 13:30	6226	3412	failure 1
sys_1	2013-11-10 14:33	3401	4902	failure 1
sys_2	2013-12-10 10:12	3401	5552	failure 2
sys_3	2013-12-10 11:40	408	4414	failure 2
⋮	⋮	⋮	⋮	⋮

Figure 2.8: Dataset of paper 5.

Event logs consist of time-ordered events; Table I shows an example of an event log file. An event is composed by a timestamp that corresponds to the time of its occurrence and other attributes that contain information about the event. These attributes may include a system or subsystem identifier that generated the event, a task id (in information system logs), description about the activity (in server logs), failure description (in hardware monitoring) and a unique event identifier. In this work we target event logs and events come from a finite and known alphabet. A timestamp and an event id are the only attributes required by our methodology.

Multiple Instance Learning (MIL) is applied when a label (class) is associated with a bag of instances (feature vectors) rather than a single instance, and it is not known in advance which subset of the vectors contributes to the class label. Similarly, in the regression formulation, a MIL setup is more appropriate: several events appear shortly before the occurrence of the target event but only a small subset of them is related to it (i.e. predictors). That is, for the interval right before the occurrence of eT where f(i;t) 0, apart from possible predictors there are other events that are not associated with eT.

stage	parameter	description
preprocessing	w	window size
	c	sigmoid shift
	s	sigmoid steepness
	γ	feature selection
model training	#trees	number of trees
	#depth	max depth of each tree
model deployment	h	decision threshold

Figure 2.9: Parameters considered in paper 5.

In the above Fig. 2.9 the authors present the list of parameters of their method. Most of the parameters can be selected via cross validation while training. Parameters w, and c should be defined by experts because they quantify the expected time (in segments) before the occurrence of eT at which a prediction is possible. The value of the decision threshold h can be configured according to desired confidence and the prediction interval.

The authors apply their method on event logs generated by monitoring systems onboard, where the events correspond to failures of aircraft components. Their goal is to predict such failures that are critical to the function of the aircraft. When these failures occur, the aircraft must stay on the ground for control or repair and therefore, being able to predict these failures in advance and plan the corresponding maintenance increases the availability of the aircraft.to create the training set for the target event eT we initially select the subset of the aircraft at which the corresponding failure occurred (ACeT AC). For the training set we select only aircraft from ACeT . We partition each sequence of events of every aircraft in ACeT into episodes and we discard the events that occurred after the last occurrence of eT of every aircraft. Our approach is leave k episodes out, that is, we randomly select k episodes from ACeT for the training set. We select half of the aircraft in ACeT for training and half for testing. In order to train the random forest model and select the proper parameters we use 5-fold cross validation on the training set

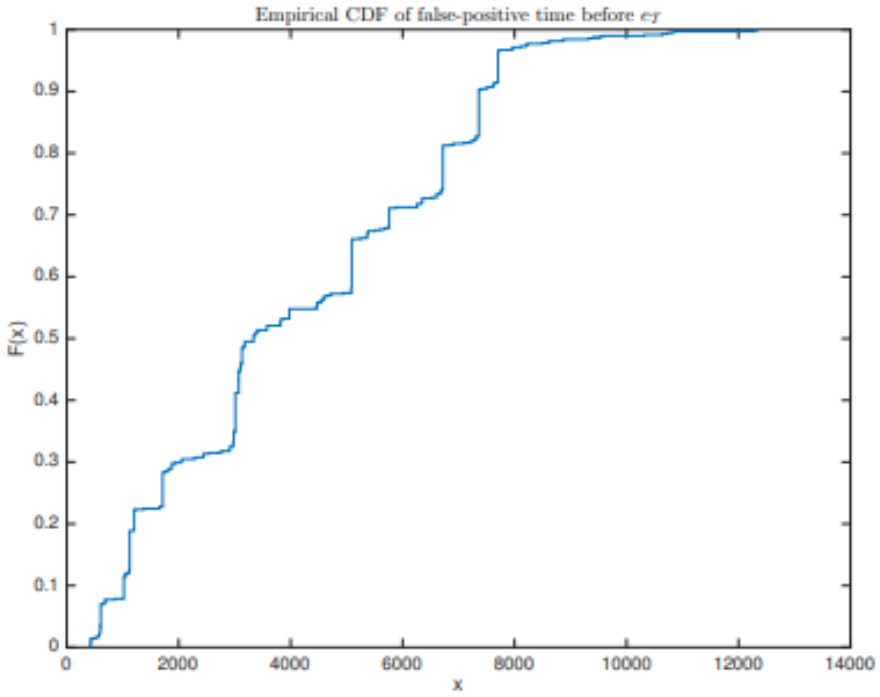

Figure 2.10: Empirical CDF of the time between false positives and eT in paper 5.

As stated previously in the paper,false positives' absence is very important in predictive maintenance and it is very critical to study their occurrence and their time of occurrence with respect to the target event. In Figure 2.10 the authors present the empirical cumulative distribution function (ecdf) of the false positives' distance from the target event.Unfortunately, a false positive may appear far before the occurrence of eT . However, a possible explanation of their presence is the intervention of engineers that affected the target event, by performing actions that prevented its occurrence.

method	precision	recall	F1-score
SVM	0.21	0.02	0.03
RFR	0.64	0.23	0.34

Figure 2.11: Prediction results at the episode level in paper 5.

The experimental evaluation presented in Fig. 2.11 shows that the method, Random Forest Regression (RFR), outperforms the selected baseline approach, which has very poor performance.In this paper the authors presented a method

for predicting future events from event logs in the context of predictive maintenance. It constitutes a novel combination of state of the art statistical and machine learning techniques and our experimental evaluation shows that it outperforms a common baseline approach. A major contribution of this work is the fact that it constitutes the first attempt to perform failure prediction given only post flight reports. Our future work will aim to increase the performance by exploiting information from a variety of sources, such as sensors, maintenance logs and environmental variables.

CHAPTER 3

System Requirement Specification

3.1 SOFTWARE REQUIREMENTS:

Operating systems: Windows 10.

Python version 3.9.4

Google Colab

3.2 HARDWARE REQUIREMENTS:

Input device (mouse / trackpad)

64 bit processor

Sufficient RAM to run the program

CHAPTER 4

Implementation

4.1 System Architecture

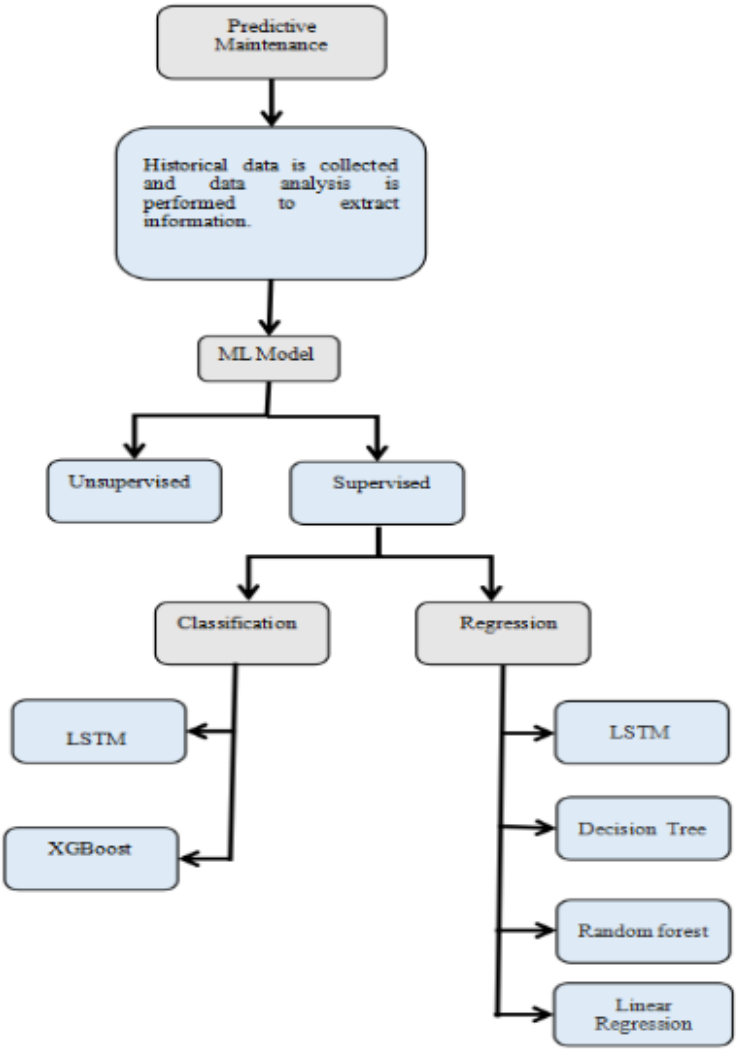

Figure 4.1: Architecture of the proposed system.

4.2 Proposed System

The project objective is to enhance the maintenance operations by applying data science techniques and machine learning algorithms for predicting more

accurate maintenance requirements.By considering the aircraft engine's sensor values over time, the machine learning algorithm can learn the relationship between the sensor values and changes in sensor values to the historical failures in order to predict failures in the future.

Some of the parameters in dataset includes:

1. aircraft engine run-to-failure events
2. operational settings
3. sensors measurements

Training data file contains aircraft engines' run-to-failure data. 20,000+ cycle records for 100 engines.

Test data file contains aircraft engines' operating data without failure events recorded. Remaining cycles for each engine are provided in a separate Ground truth data file.

Ground truth data file contains the true remaining cycles for each engine in the test data. Exploratory Data Analysis: Feature variability, distribution, and correlation were examined to uncover underlying structure and extract important variables.

	id	cycle	setting1	setting2	setting3	s1	s2	s3	s4	s5	...	s12	s13	s14	s15	s16	s17	s18	s19	s20	s21
0	1	1	-0.0007	-0.0004	100.0	518.67	641.82	1589.70	1400.60	14.62	...	521.66	2388.02	8138.62	8.4195	0.03	392	2388	100.0	39.06	23.4190
1	1	2	0.0019	-0.0003	100.0	518.67	642.15	1591.82	1403.14	14.62	...	522.28	2388.07	8131.49	8.4318	0.03	392	2388	100.0	39.00	23.4236
2	1	3	-0.0043	0.0003	100.0	518.67	642.35	1587.99	1404.20	14.62	...	522.42	2388.03	8133.23	8.4178	0.03	390	2388	100.0	38.95	23.3442
3	1	4	0.0007	0.0000	100.0	518.67	642.35	1582.79	1401.87	14.62	...	522.86	2388.08	8133.83	8.3682	0.03	392	2388	100.0	38.88	23.3739
4	1	5	-0.0019	-0.0002	100.0	518.67	642.37	1582.85	1406.22	14.62	...	522.19	2388.04	8133.80	8.4294	0.03	393	2388	100.0	38.90	23.4044

Figure 4.2: Dataset 1-Snippet of Training data

	id	cycle	setting1	setting2	setting3	s1	s2	s3	s4	s5	...	s12	s13	s14	s15	s16	s17	s18	s19	s20	s21
0	1	1	0.0023	0.0003	100.0	518.67	643.02	1585.29	1398.21	14.62	...	521.72	2388.03	8125.55	8.4052	0.03	392	2388	100.0	38.86	23.3735
1	1	2	-0.0027	-0.0003	100.0	518.67	641.71	1588.45	1395.42	14.62	...	522.16	2388.06	8139.62	8.3803	0.03	393	2388	100.0	39.02	23.3916
2	1	3	0.0003	0.0001	100.0	518.67	642.46	1586.94	1401.34	14.62	...	521.97	2388.03	8130.10	8.4441	0.03	393	2388	100.0	39.08	23.4166
3	1	4	0.0042	0.0000	100.0	518.67	642.44	1584.12	1406.42	14.62	...	521.38	2388.05	8132.90	8.3917	0.03	391	2388	100.0	39.00	23.3737
4	1	5	0.0014	0.0000	100.0	518.67	642.51	1587.19	1401.92	14.62	...	522.15	2388.03	8129.54	8.4031	0.03	390	2388	100.0	38.99	23.4130

Figure 4.3: Dataset 1-Snippet of Testing data

	id	cycle	setting1	setting2	setting3	s1	s2	s3	s4	s5	...	s12	s13	s14	s15	s16	s17	s18	s19	s20	s21
0	1	1	34.9983	0.8400	100.0	449.44	555.32	1358.61	1137.23	5.48	...	183.06	2387.72	8048.56	9.3461	0.02	334	2223	100.00	14.73	8.8071
1	1	2	41.9982	0.8408	100.0	445.00	549.90	1353.22	1125.78	3.91	...	130.42	2387.66	8072.30	9.3774	0.02	330	2212	100.00	10.41	6.2665
2	1	3	24.9988	0.6218	60.0	462.54	537.31	1256.76	1047.45	7.05	...	164.22	2028.03	7864.87	10.8941	0.02	309	1915	84.93	14.08	8.6723
3	1	4	42.0077	0.8416	100.0	445.00	549.51	1354.03	1126.38	3.91	...	130.72	2387.61	8068.66	9.3528	0.02	329	2212	100.00	10.59	6.4701
4	1	5	25.0005	0.6203	60.0	462.54	537.07	1257.71	1047.93	7.05	...	164.31	2028.00	7861.23	10.8963	0.02	309	1915	84.93	14.13	8.5286

Figure 4.4: Dataset 2-Snippet of the Training data

	id	cycle	setting1	setting2	setting3	s1	s2	s3	s4	s5	...	s12	s13	s14	s15	s16	s17	s18	s19	s20	s21
0	1	1	9.9987	0.2502	100.0	489.05	605.03	1497.17	1304.99	10.52	...	371.69	2388.18	8114.10	8.6476	0.03	369	2319	100.00	28.42	17.1551
1	1	2	20.0026	0.7000	100.0	491.19	607.82	1481.20	1246.11	9.35	...	315.32	2388.12	8053.06	9.2405	0.02	364	2324	100.00	24.29	14.8039
2	1	3	35.0045	0.8400	100.0	449.44	556.00	1359.08	1128.36	5.48	...	183.04	2387.75	8053.04	9.3472	0.02	333	2223	100.00	14.98	8.9125
3	1	4	42.0066	0.8410	100.0	445.00	550.17	1349.69	1127.89	3.91	...	130.40	2387.72	8066.90	9.3961	0.02	332	2212	100.00	10.35	6.4181
4	1	5	24.9985	0.6213	60.0	462.54	536.72	1253.18	1050.69	7.05	...	164.56	2028.05	7865.66	10.8682	0.02	305	1915	84.93	14.31	8.5740

Figure 4.5: Dataset 2-Snippet of the Testing data

4.3 Data Preprocessing

We performed the following steps:

1.Data Loading

2.Missing Values Identification

```
id          0
cycle       0
setting1    0
setting2    0
setting3    0
s1          0
s2          0
s3          0
s4          0
s5          0
s6          0
s7          0
s8          0
s9          0
s10         0
s11         0
s12         0
s13         0
s14         0
s15         0
s16         0
s17         0
s18         0
s19         0
s20         0
s21         0
dtype: int64
```

Figure 4.6: Missing values identification

3.Feature extraction:It is applied to the training and test data by introducing additional two columns for each of the 21 sensor columns which is rolling mean and rolling standard deviation.

	s12	s7	s21	s20	s6	s14	s9	s13	s8	s3
count	20631.000000	20631.000000	20631.000000	20631.000000	20631.000000	20631.000000	20631.000000	20631.000000	20631.000000	20631.000000
mean	521.413470	553.367711	23.289705	38.816271	21.609803	8143.752722	9065.242941	2388.096152	2388.096652	1590.523119
std	0.737553	0.885092	0.108251	0.180746	0.001389	19.076176	22.082880	0.071919	0.070985	6.131150
min	518.690000	549.850000	22.894200	38.140000	21.600000	8099.940000	9021.730000	2387.880000	2387.900000	1571.040000
25%	520.960000	552.810000	23.221800	38.700000	21.610000	8133.245000	9053.100000	2388.040000	2388.050000	1586.260000
50%	521.480000	553.440000	23.297900	38.830000	21.610000	8140.540000	9060.660000	2388.090000	2388.090000	1590.100000
75%	521.950000	554.010000	23.366800	38.950000	21.610000	8148.310000	9069.420000	2388.140000	2388.140000	1594.380000
max	523.380000	556.060000	23.618400	39.430000	21.610000	8293.720000	9244.590000	2388.560000	2388.560000	1616.910000

Figure 4.7: Feature extraction

4.4 Algorithms: Modeling strategies used for predictive maintenance of aircraft engines are:

Regression Modeling algorithms-Predicting Engine's Remaining Useful Lifetime of the engine (RUL):

To predict the number of remaining cycles before engine failure.Both linear and non-linear regression models are implemented to predict engine's RUL.The following machine learning algorithms and their performance metrics were calculated and evaluated namely:

1.LSTM

2.Multiple Linear Regression

3.Decision Tree Regression

4.Random Forests

1.LSTM

Long Short Term Memory networks usually known as "LSTMs" – are a special kind of RNN, capable of learning long-term dependencies. They work tremendously well on a large variety of problems, and are now widely used.LSTMs are explicitly designed to avoid the long-term dependency problem. Remembering information for long periods of time is practically their default behavior, not something they struggle to learn.

LSTMs also have chain like structure, but the repeating module has a different structure. Instead of having a single neural network layer, there are four, interacting in a very special way.The key to LSTMs is the cell state.The LSTM does have the ability to remove or add information to the cell state,

carefully regulated by structures called gates.Gates are a way to optionally let information through.

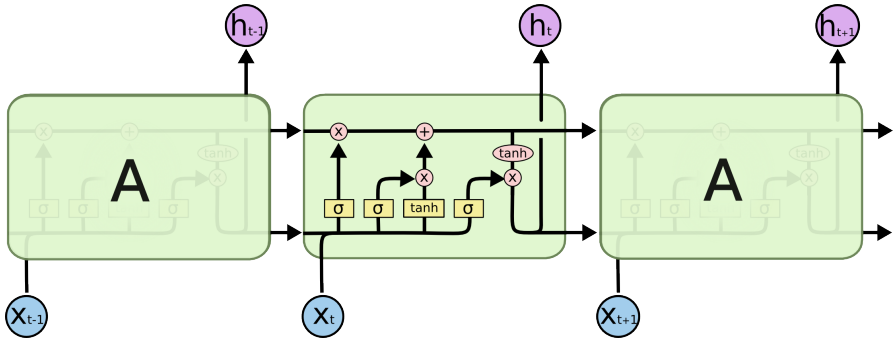

Figure 4.8: Block Diagram of LSTM

First step is to generate labels for the training data which are Remaining Useful Life (RUL), label1 and label2 .cycle column is also used for training so,we normalized the columns in the training dataset and testing dataset.LSTMs are able to use larger window sizes and use all the information in the window as input.In this algorithm we considered the window size as 50.

Keras LSTM layers expect an input in the shape of a numpy array of 3 dimensions (samples, time steps, features) where samples is the number of training sequences, time steps is the look back window or sequence length and features is the number of features of each sequence at each time step.Next, we build a deep network. The first layer is an LSTM layer with 100 units followed by another LSTM layer with 50 units. Dropout is also applied after each LSTM layer to control overfitting. Final layer is a Dense output layer with single unit and linear activation.

```python
def r2_keras(y_true, y_pred):
    """Coefficient of Determination
    """
    SS_res =  K.sum(K.square( y_true - y_pred ))
    SS_tot = K.sum(K.square( y_true - K.mean(y_true) ) )
    return ( 1 - SS_res/(SS_tot + K.epsilon()) )


nb_features = seq_array.shape[2]
nb_out = label_array.shape[1]

model = Sequential()
model.add(LSTM(
         input_shape=(sequence_length, nb_features),
         units=100,
         return_sequences=True))
model.add(Dropout(0.2))
model.add(LSTM(
          units=50,
          return_sequences=False))
model.add(Dropout(0.2))
model.add(Dense(units=nb_out))
model.add(Activation("linear"))
model.compile(loss='mean_squared_error', optimizer='rmsprop',metrics=['mae',r2_keras])

print(model.summary())

# fit the network
history = model.fit(seq_array, label_array, epochs=100, batch_size=200, validation_split=0.05, verbose=2,
          callbacks = [keras.callbacks.EarlyStopping(monitor='val_loss', min_delta=0, patience=10, verbose=0, mode='min'),
                       keras.callbacks.ModelCheckpoint(model_path,monitor='val_loss', save_best_only=True, mode='min', verbose=0)]
          )

print(history.history.keys())
```

Figure 4.9: LSTM Algorithm

```
75/75 - 16s - loss: 740.8986 - mae: 17.4587 - r2_keras: 0.7731 - val_loss: 646.4289 - val_mae: 16.6409 - val_r2_keras: 0.7162 - 16s/epoch - 214ms/step
Epoch 56/100
75/75 - 16s - loss: 743.2350 - mae: 17.5586 - r2_keras: 0.7739 - val_loss: 707.9951 - val_mae: 18.9491 - val_r2_keras: 0.6630 - 16s/epoch - 215ms/step
Epoch 57/100
75/75 - 16s - loss: 733.8604 - mae: 17.3250 - r2_keras: 0.7745 - val_loss: 781.1907 - val_mae: 20.0515 - val_r2_keras: 0.6216 - 16s/epoch - 215ms/step
Epoch 58/100
75/75 - 16s - loss: 721.8291 - mae: 17.1818 - r2_keras: 0.7792 - val_loss: 664.0344 - val_mae: 17.0945 - val_r2_keras: 0.7238 - 16s/epoch - 213ms/step
Epoch 59/100
75/75 - 16s - loss: 700.6791 - mae: 16.9973 - r2_keras: 0.7850 - val_loss: 737.0256 - val_mae: 16.7626 - val_r2_keras: 0.7409 - 16s/epoch - 213ms/step
Epoch 60/100
75/75 - 16s - loss: 696.4380 - mae: 16.9383 - r2_keras: 0.7869 - val_loss: 812.8745 - val_mae: 16.6415 - val_r2_keras: 0.7140 - 16s/epoch - 213ms/step
Epoch 61/100
75/75 - 16s - loss: 690.6730 - mae: 16.8026 - r2_keras: 0.7893 - val_loss: 625.0580 - val_mae: 15.2251 - val_r2_keras: 0.7404 - 16s/epoch - 215ms/step
Epoch 62/100
75/75 - 16s - loss: 673.6978 - mae: 16.6385 - r2_keras: 0.7923 - val_loss: 825.8182 - val_mae: 19.1754 - val_r2_keras: 0.5472 - 16s/epoch - 215ms/step
Epoch 63/100
75/75 - 16s - loss: 663.2423 - mae: 16.4758 - r2_keras: 0.7964 - val_loss: 702.1575 - val_mae: 18.0680 - val_r2_keras: 0.7084 - 16s/epoch - 216ms/step
Epoch 64/100
75/75 - 16s - loss: 655.8093 - mae: 16.4096 - r2_keras: 0.7985 - val_loss: 834.7640 - val_mae: 18.6589 - val_r2_keras: 0.5738 - 16s/epoch - 214ms/step
Epoch 65/100
75/75 - 16s - loss: 635.6663 - mae: 16.1783 - r2_keras: 0.8056 - val_loss: 736.1179 - val_mae: 15.3038 - val_r2_keras: 0.6791 - 16s/epoch - 214ms/step
Epoch 66/100
75/75 - 16s - loss: 627.4375 - mae: 16.0963 - r2_keras: 0.8081 - val_loss: 718.3407 - val_mae: 17.7013 - val_r2_keras: 0.6403 - 16s/epoch - 212ms/step
Epoch 67/100
75/75 - 16s - loss: 625.4490 - mae: 16.0426 - r2_keras: 0.8075 - val_loss: 887.7797 - val_mae: 17.6202 - val_r2_keras: 0.6540 - 16s/epoch - 214ms/step
Epoch 68/100
75/75 - 16s - loss: 620.5270 - mae: 15.8862 - r2_keras: 0.8104 - val_loss: 791.1064 - val_mae: 18.9551 - val_r2_keras: 0.5557 - 16s/epoch - 212ms/step
Epoch 69/100
75/75 - 16s - loss: 607.0981 - mae: 15.7673 - r2_keras: 0.8143 - val_loss: 907.6631 - val_mae: 17.5356 - val_r2_keras: 0.6544 - 16s/epoch - 212ms/step
Epoch 70/100
75/75 - 16s - loss: 591.0731 - mae: 15.5356 - r2_keras: 0.8196 - val_loss: 842.3083 - val_mae: 16.1227 - val_r2_keras: 0.6900 - 16s/epoch - 213ms/step
Epoch 71/100
75/75 - 16s - loss: 577.5673 - mae: 15.4935 - r2_keras: 0.8236 - val_loss: 857.7706 - val_mae: 19.4100 - val_r2_keras: 0.5345 - 16s/epoch - 217ms/step
dict_keys(['loss', 'mae', 'r2_keras', 'val_loss', 'val_mae', 'val_r2_keras'])
```

Figure 4.10: Training the Model

2.Multiple Linear Regression

A statistical method of analysis that estimates the relationship between one or more independent variables and a dependent variable; the method estimates the relationship by minimizing the sum of the squares in the difference between the observed and predicted values of the dependent variable configured as a straight line or hyperplane.Multiple regression is like linear regression, but with more than one independent value, meaning that we try to predict a value based on two or more variables.

A.The Cost Function

Multiple Linear Regression computes cost function by calculating a metric known as cost, which is the degree of error between the hyperplane's values

and those of the training dataset. The cost can be calculated by many different formulas, but the one that linear regression uses is known as the multivariate Mean Squared Error (MSE) Cost Function. By determining how well a hyperplane represents the data, the regression model can tune the values of the parameters and optimize accuracy.

B.Gradient Descent Gradient descent is a complex algorithm.Because of its complexity,after completing gradient descent, our algorithm will have finalized all the parameter values and created the equation of the optimal hyperplane. This equation can then be used to predict future values.

C.Prediction

From the sklearn module we can use the LinearRegression() method to create a linear regression object. This object has a method called fit() that takes the independent and dependent values as parameters and fills the regression object with data that describes the relationship.Code snippet is shown in Fig 4.11

```
#multiple linear regression

linreg = linear_model.LinearRegression()
linreg.fit(X_train, y_train)

y_test_predict = linreg.predict(X_test)
y_train_predict = linreg.predict(X_train)


linreg_metrics = get_regression_metrics('Multiple Linear Regression', y_test, y_test_predict)
linreg_metrics
```

Figure 4.11: Code snippet of Multiple Linear Regression

3.Decision Tree Regression

It learns in a hierarchical fashion by repeatedly splitting the dataset into separate branches that maximize the information gain of each split. In regression tree, the value obtained by terminal nodes in the training data is the mean response of observation falling in that region.The branches/edges represent the result of the node and the nodes have either:

1.Conditions [Decision Nodes]

2.Result [End Nodes]

The branches/edges represent the truth/falsity of the statement and take makes a decision based on that in the example below which shows a decision tree that evaluates the smallest of three numbers.Decision tree regression

observes features of an object and trains a model in the structure of a tree to predict data in the future to produce meaningful continuous output. Continuous output means that the output/result is not discrete, i.e., it is not represented just by a discrete, known set of numbers or values.Entropy basically measures the impurity of a node. Impurity is the degree of randomness; it tells how random our data is. A pure sub-split means that either you should be getting "yes", or you should be getting "no".

Information gain measures the reduction of uncertainty given some feature and it is also a deciding factor for which attribute should be selected as a decision node or root node.Pruning It is another method that can help us avoid overfitting. It helps in improving the performance of the tree by cutting the nodes or sub-nodes which are not significant. It removes the branches which have very low importance.

```
#Decision Tree regressor

#dtrg = DecisionTreeRegressor(max_depth=8, max_features=5, random_state=123) # selected fea
dtrg = DecisionTreeRegressor(max_depth=7, random_state=123)
dtrg.fit(X_train, y_train)

y_test_predict = dtrg.predict(X_test)
y_train_predict = dtrg.predict(X_train)


dtrg_metrics = get_regression_metrics('Decision Tree Regression', y_test, y_test_predict)
dtrg_metrics
```

Figure 4.12: Code snippet of Decision Tree Regression

4.Random Forest

An ensemble learning method for classification, regression and other tasks, that operate by constructing a multitude of decision trees at training time and outputting the class that is mean prediction (regression) of the individual trees.The bootstrapping Random Forest algorithm combines ensemble learning methods with the decision tree framework to create multiple randomly drawn decision trees from the data, averaging the results to output a new result that often leads to strong predictions.

We use the sklearn module for training our random forest regression model, specifically the RandomForestRegressor function. The RandomForestRegressor documentation shows many different parameters we can select for our model.

```
#Random Forest

#rf = RandomForestRegressor(n_estimators=100, max_features=2, max_depth=4, n_jobs=-1, random_state=1)
rf = RandomForestRegressor(n_estimators=100, max_features=3, max_depth=4, n_jobs=-1, random_state=1)
#rf = RandomForestRegressor(n_estimators=100, max_features=3, max_depth=7, n_jobs=-1, random_state=1)

rf.fit(X_train, y_train)

y_test_predict = rf.predict(X_test)
y_train_predict = rf.predict(X_train)



rf_metrics = get_regression_metrics('Random Forest Regression', y_test, y_test_predict)
rf_metrics
```

Figure 4.13: Code snippet of Random Forest

Binary Classification algorithms-Engines failure in a current period:

To predict if the engine will fail within a specific cycle window or not is determined by binary classification algorithm.Binary classification algorithms are implemented to predict which engines will fail in the current period, e.g. remaining cycles or TTF in the range 0-30 cycles.The following machine learning algorithms and their performance metrics were calculated and evaluated namely:

1.LSTM

2.XGBoost

1.LSTM

Long Short Term Memory networks usually known as "LSTMs" – are a special kind of RNN, capable of learning long-term dependencies. They work tremendously well on a large variety of problems, and are now widely used.LSTMs are explicitly designed to avoid the long-term dependency problem. Remembering information for long periods of time is practically their default behavior, not something they struggle to learn.

LSTMs also have chain like structure, but the repeating module has a different structure. Instead of having a single neural network layer, there are four, interacting in a very special way.The key to LSTMs is the cell state.The LSTM does have the ability to remove or add information to the cell state, carefully regulated by structures called gates.Gates are a way to optionally let information through. They are composed out of a sigmoid neural net layer and a pointwise multiplication operation.

We can train a deep neural network to classify sequence data, by an LSTM

network. An LSTM network enables to input sequence data into a network, and make predictions based on the individual time steps of the sequence data.Long short-term memory (LSTM) is a deep recurrent neural network architecture used for classification of time-series data.Recurrent Neural networks like LSTM generally have the problem of overfitting. Dropout can be applied between layers using the Dropout Keras laye to avoid over-fitting.

```
# read training data - It is the aircraft engine run-to-failure data.
train_df = pd.read_csv('/content/drive/MyDrive/Project Seminar/Data/data1/PM_train.txt', sep=' ', header=None)
train_df.drop(train_df.columns[[26, 27]], axis=1, inplace=True)
train_df.columns = ['id', 'cycle', 'setting1', 'setting2', 'setting3', 's1', 's2', 's3',
                    's4', 's5', 's6', 's7', 's8', 's9', 's10', 's11', 's12', 's13', 's14',
                    's15', 's16', 's17', 's18', 's19', 's20', 's21']

train_df = train_df.sort_values(['id','cycle'])

# read test data - It is the aircraft engine operating data without failure events recorded.
test_df = pd.read_csv('/content/drive/MyDrive/Project Seminar/Data/data1/PM_test.txt', sep=" ", header=None)
test_df.drop(test_df.columns[[26, 27]], axis=1, inplace=True)
test_df.columns = ['id', 'cycle', 'setting1', 'setting2', 'setting3', 's1', 's2', 's3',
                    's4', 's5', 's6', 's7', 's8', 's9', 's10', 's11', 's12', 's13', 's14',
                    's15', 's16', 's17', 's18', 's19', 's20', 's21']

# read ground truth data - It contains the information of true remaining cycles for each engine in the testing data.
truth_df = pd.read_csv('/content/drive/MyDrive/Project Seminar/Data/data1/PM_truth.txt', sep=" ", header=None)
truth_df.drop(truth_df.columns[[1]], axis=1, inplace=True)
```

Figure 4.14: LSTM data ingestion

which are Remaining Useful Life (RUL), label1 and label2.cycle column is also used for training so we will also include the cycle column. Here, we normalize the columns in the training data.Next, we prepare the test data. We first normalize the test data using the parameters from the MinMax normalization applied on the training data.Next, we use the ground truth dataset to generate labels for the test data.

Data Preprocessing

```
# TRAIN

rul = pd.DataFrame(train_df.groupby('id')['cycle'].max()).reset_index()
rul.columns = ['id', 'max']
train_df = train_df.merge(rul, on=['id'], how='left')
train_df['RUL'] = train_df['max'] - train_df['cycle']
train_df.drop('max', axis=1, inplace=True)


w1 = 30
w0 = 15
train_df['label1'] = np.where(train_df['RUL'] <= w1, 1, 0 )
train_df['label2'] = train_df['label1']
train_df.loc[train_df['RUL'] <= w0, 'label2'] = 2
```

Figure 4.15: Generation of RUL for train data in LSTM

```
[ ]  # MinMax normalization (from 0 to 1)
     train_df['cycle_norm'] = train_df['cycle']
     cols_normalize = train_df.columns.difference(['id','cycle','RUL','label1','label2'])
     min_max_scaler = preprocessing.MinMaxScaler()

     norm_train_df = pd.DataFrame(min_max_scaler.fit_transform(train_df[cols_normalize]),
                                  columns=cols_normalize,
                                  index=train_df.index)
     join_df = train_df[train_df.columns.difference(cols_normalize)].join(norm_train_df)
     train_df = join_df.reindex(columns = train_df.columns)
```

Figure 4.16: MinMax Normalization for train data

```
# generate RUL for test data
test_df = test_df.merge(truth_df, on=['id'], how='left')
test_df['RUL'] = test_df['max'] - test_df['cycle']
test_df.drop('max', axis=1, inplace=True)

# generate label columns w0 and w1 for test data
test_df['label1'] = np.where(test_df['RUL'] <= w1, 1, 0 )
test_df['label2'] = test_df['label1']
test_df.loc[test_df['RUL'] <= w0, 'label2'] = 2
```

Figure 4.17: Generation of RUL for test data in LSTM

```
# TEST
######
# MinMax normalization (from 0 to 1)
test_df['cycle_norm'] = test_df['cycle']
norm_test_df = pd.DataFrame(min_max_scaler.transform(test_df[cols_normalize]),
                            columns=cols_normalize,
                            index=test_df.index)
test_join_df = test_df[test_df.columns.difference(cols_normalize)].join(norm_test_df)
test_df = test_join_df.reindex(columns = test_df.columns)
test_df = test_df.reset_index(drop=True)
print(test_df.head())


rul = pd.DataFrame(test_df.groupby('id')['cycle'].max()).reset_index()
rul.columns = ['id', 'max']
truth_df.columns = ['more']
truth_df['id'] = truth_df.index + 1
truth_df['max'] = rul['max'] + truth_df['more']
truth_df.drop('more', axis=1, inplace=True)
```

Figure 4.18: Minmax Normalization for test data

One critical advantage of LSTMs is their ability to remember from long-term sequences (window sizes) which is hard to achieve by traditional feature engineering. LSTMs are able to use larger window sizes and use all the information in the window as input. Keras LSTM layers expect an input in the shape of a numpy array of 3 dimensions (samples, time steps, features) where samples is the number of training sequences, time steps is the look back window or sequence length and features is the number of features of each sequence at each time step.LSTM Network

```
nb_features = seq_array.shape[2]
nb_out = label_array.shape[1]

model = Sequential()
model.add(LSTM(input_shape=(sequence_length, nb_features), units=100, return_sequences=True))
model.add(Dropout(0.2))
model.add(LSTM(units=100, return_sequences=False))
model.add(Dropout(0.2))
model.add(Dense(units=nb_out, activation='sigmoid'))

model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
```

Figure 4.19: Model of LSTM

In the proposed system for classification,the first layer is an LSTM layer with 100 units followed by another LSTM layer with 50 units. Dropout is also applied after each LSTM layer to control overfitting. Final layer is a Dense output layer with single unit and sigmoid activation since this is a binary classification problem.

```
%%time
model.fit(seq_array, label_array, epochs=10, batch_size=200, validation_split=0.05, verbose=1,
          callbacks = [keras.callbacks.EarlyStopping(monitor='val_loss', min_delta=0, patience=0, verbose=0, mode='auto')])

Epoch 1/10
75/75 [==============================] - 33s 378ms/step - loss: 0.2459 - accuracy: 0.8906 - val_loss: 0.0669 - val_accuracy: 0.9872
Epoch 2/10
75/75 [==============================] - 27s 358ms/step - loss: 0.1008 - accuracy: 0.9579 - val_loss: 0.0788 - val_accuracy: 0.9616
CPU times: user 1min 34s, sys: 5.64 s, total: 1min 40s
Wall time: 59.7 s
<keras.callbacks.History at 0x7f98b2680e50>
```

Figure 4.20: Training the LSTM Model

2.XGBoost :XGBoost classifier is a Machine learning algorithm that is applied for structured and tabular data. XGBoost is an implementation of gradient boosted decision trees designed for speed and performance. XGBoost is an extreme gradient boost algorithm. And that means it's a big Machine learning algorithm with lots of parts.XGBoost is a great starting point for machine learning on structured (tabular) data. It usually offers an attractive combination of great prediction performance and low processing time. Another plus is that the algorithm gracefully handles nulls: even after our preprocessing we still have a lot of missing values.

```
# read training data - It is the aircraft engine run-to-failure data.
train_df = pd.read_csv('/content/drive/MyDrive/Project Seminar/Data/data1/PM_train.txt', sep=' ', header=None)
train_df.drop(train_df.columns[[26, 27]], axis=1, inplace=True)
train_df.columns = ['id', 'cycle', 'setting1', 'setting2', 'setting3', 's1', 's2', 's3',
                    's4', 's5', 's6', 's7', 's8', 's9', 's10', 's11', 's12', 's13', 's14',
                    's15', 's16', 's17', 's18', 's19', 's20', 's21']

train_df = train_df.sort_values(['id','cycle'])

# read test data - It is the aircraft engine operating data without failure events recorded.
test_df = pd.read_csv('/content/drive/MyDrive/Project Seminar/Data/data1/PM_test.txt', sep=" ", header=None)
test_df.drop(test_df.columns[[26, 27]], axis=1, inplace=True)
test_df.columns = ['id', 'cycle', 'setting1', 'setting2', 'setting3', 's1', 's2', 's3',
                   's4', 's5', 's6', 's7', 's8', 's9', 's10', 's11', 's12', 's13', 's14',
                   's15', 's16', 's17', 's18', 's19', 's20', 's21']

# read ground truth data - It contains the information of true remaining cycles for each engine in the testing data.
truth_df = pd.read_csv('/content/drive/MyDrive/Project Seminar/Data/data1/PM_truth.txt', sep=" ", header=None)
truth_df.drop(truth_df.columns[[1]], axis=1, inplace=True)
```

Figure 4.21: XGBoost data ingestion

we ingest the training, test and ground truth datasets.First step is to generate labels for the training data

Generate Labels for Train Data

```
# Data Labeling - generating column RUL
rul = pd.DataFrame(train_df.groupby('id')['cycle'].max()).reset_index()
rul.columns = ['id', 'max']
train_df = train_df.merge(rul, on=['id'], how='left')
train_df['RUL'] = train_df['max'] - train_df['cycle']
train_df.drop('max', axis=1, inplace=True)
train_df.head()
```

Figure 4.22: Generation of RUL label for Train data

```
[ ] # generating label columns for training data
    w1 = 30
    w0 = 24
    train_df['label1'] = np.where(train_df['RUL'] <= w1, 1, 0 )
    train_df['label2'] = train_df['label1']
    train_df.loc[train_df['RUL'] <= w0, 'label2'] = 2
    train_df.head()
```

Figure 4.23: Generation of label1 and label2 for Train data

Generate Labels for Test Data

```
[ ] # generate column max for test data
    rul = pd.DataFrame(test_df.groupby('id')['cycle'].max()).reset_index()
    rul.columns = ['id', 'max']
    truth_df.columns = ['more']
    truth_df['id'] = truth_df.index + 1
    truth_df['max'] = rul['max'] + truth_df['more']
    truth_df.drop('more', axis=1, inplace=True)
```

Figure 4.24: Generation of max label for Test data

```
[ ]  # generate RUL for test data
     test_df = test_df.merge(truth_df, on=['id'], how='left')
     test_df['RUL'] = test_df['max'] - test_df['cycle']
     test_df.drop('max', axis=1, inplace=True)
     test_df.head()
```

Figure 4.25: Generation of RUL label for Test data

```
[ ]  # generate label columns w0 and w1 for test data
     test_df['label1'] = np.where(test_df['RUL'] <= w1, 1, 0 )
     test_df['label2'] = test_df['label1']
     test_df.loc[test_df['RUL'] <= w0, 'label2'] = 2
     test_df.head()
```

Figure 4.26: Generation of label1,label2 for Train data

This algorithm uses multiple parameters The overall parameters have been divided into 3 categories: 1.General Parameters: Guide the overall functioning 2.Booster Parameters: Guide the individual booster (tree/regression) at each step 3.Learning Task Parameters: Guide the optimization performed

```
import xgboost as xgb

params = {}
params['booster'] = 'gbtree'
params['objective'] = 'binary:logistic'
params['eta'] = 0.006
params['eval_metric'] = 'auc'
params['max_depth'] = 4
params['colsample_bytree'] = 0.6
params['subsample'] = 0.5
params['silent'] = 1

d_train = xgb.DMatrix(X_train, label=Y_train)
d_valid = xgb.DMatrix(X_val, label=Y_val)
watchlist = [(d_train, 'train'), (d_valid, 'valid')]

gbm = xgb.train(params, d_train, 2000, watchlist, early_stopping_rounds=50, verbose_eval=25)
```

Figure 4.27: XGBoost Algorithm

CHAPTER 5

Results

For predicting Remaining Useful Life(RUL),different algorithms' performance were compared and evaluated on the dataset1 and dataset2.So,for predicting RUL,the main goal is to reduce the error value of the predicted RUL when compared with the actual RUL. For every dataset, the results of the testset were compared with the actual RUL values in the data set.The algorithms were evaluated on their mean absolute error (MAE),a comparison of their performance is in Fig. 5.5 and 5.6.The residual graphs of linear regression as shown in Fig. 5.1,random forest as shown in Fig. 5.2 and decision tree as shown in Fig. 5.3 are plotted.For LSTM algorithm MAE is plotted against the number of epochs in Fig. 5.4 and it is observed that as no. of epochs are increasing the error is decreasing.It is observed that the best results were obtained by LSTM algorithm.

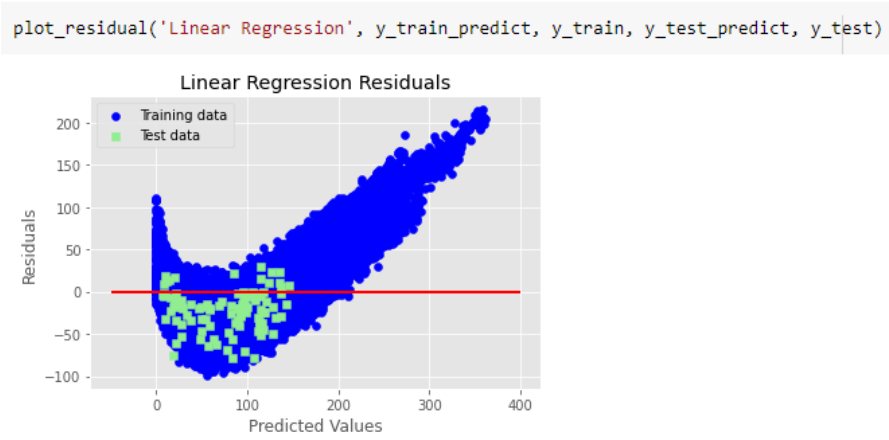

Figure 5.1: Output graph of Multilinear regression

```
plot_residual('Random Forest Regression', y_train_predict, y_train, y_test_predict, y_test)
```

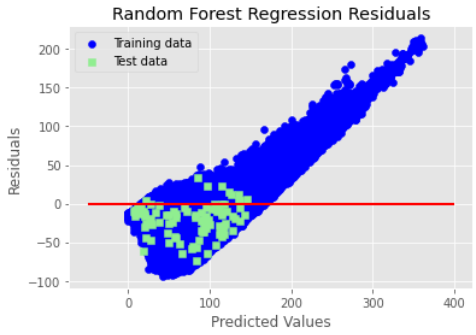

Figure 5.2: Output graph of Random forest regression

```
plot_residual('Decision Tree Regression', y_train_predict, y_train, y_test_predict, y_test)
```

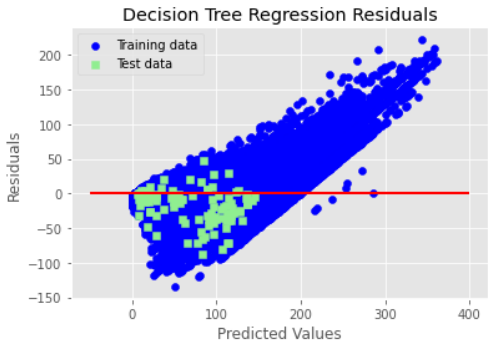

Figure 5.3: Output graph of decision tree regression

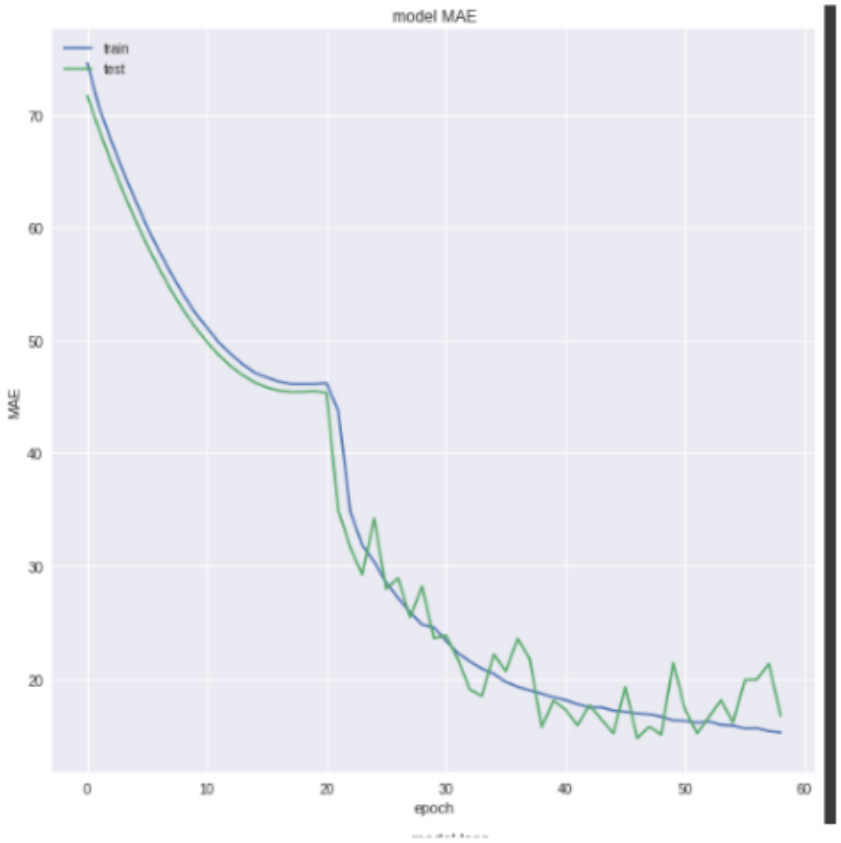

Figure 5.4: Graph of LSTM

Comparison of different algorithms for RUL on dataset 1

ALGORITHMS	MEAN ABSOLUTE ERROR(MAE)
LSTM	14.22
Decision Tree	24.32
Random forest	23.17
Linear Regression	25.59

Figure 5.5: Comparision of MAE on dataset 1

Comparison of different algorithms for RUL on dataset 2

ALGORITHMS	MEAN ABSOLUTE ERROR(MAE)
LSTM	20.1
Decision Tree	26.3
Random forest	35.3
Linear Regression	27.4

Figure 5.6: Comparision of MAE on dataset 2

For predicting if the engine will fail within a specific cycle window or not,different algorithms' performance were compared and evaluated on the dataset.The algorithms were evaluated on their accuracy,a comparison of their performance is depicted in Fig. 5.7 and Fig. 5.8.It is observed that the best results were obtained by LSTM algorithm

```
results_df = pd.DataFrame([[scores_test[1],precision_test,recall_test,f1_test]],
                          columns = ['Accuracy', 'Precision', 'Recall', 'F1-score'],
                          index = ['LSTM'])
results_df
```

	Accuracy	Precision	Recall	F1-score
LSTM	0.956989	1.0	0.84	0.913043

Figure 5.7: LSTM Output

```
[ ] results_df = pd.DataFrame([[acc,precision_test,recall_test,f1_test],
                           ],
                          columns = ['Accuracy', 'Precision', 'Recall', 'F1-score'],
                          index = ['XGBoost'])
    results_df
```

	Accuracy	Precision	Recall	F1-score
XGBoost	0.91	0.944444	0.68	0.790698

Figure 5.8: XGBoost Output

CHAPTER 6

Conclusions and Future Scope

Predictive Maintenance of Aircraft Engines detects the anomalies and failure patterns and provides early warnings. By applying machine learning algorithms to historical dataset of engines,the project can predict rul and engine fails or not for aircraft engines. Among various algorithms applied on the datasets Long Short Term Memory(LSTM) has given better results for MAE and accuracy for predicting RUL and engine failure respectively.

The future scope of this project is to develop an interactive UI for loading the dataset of any aircraft engine to find out the Rmaining Useful Life(RUL,to display the RUL of any engine selected by assuming the historical data of aircraft engines and degradation pattern reflected in sensor values.The UI should also display the health state of any engine selected,so as to let know the engineer in a particular period whether it fails or not.

References

@onlineberlin, title = Berlin Cathedral, url = https://en.wikipedia.org/wiki/Berlin$_Cathedr$

@article avidanshai, author = "Hermawan, Ade Pitra, Dong-Seong Kim, and Jae-Min Lee", title = "Predictive Maintenance of Aircraft Engine using Deep Learning Technique.", journal = "IEEE", pages = "1296-1298", year = "2020"

@article Xie, author = "Xia, Pengcheng, Yixiang Huang, Peng Li, Chengliang Liu, and Lun Shi", title = "Fault Knowledge Transfer Assisted Ensemble Method for Remaining Useful Life Prediction.", journal = " IEEE Transactions on Industrial Informatics 18,no. 3", pages = "1758-1769", year = "2021"

@article andrewng, author = "Singh, Deepankar, Mithilesh Kumar, K. V. Arya, and Sunil Kumar", title = "Aircraft Engine Reliability Analysis using Machine Learning Algorithms.", journal = "In 2020 IEEE 15th International Conference on Industrial and Information Systems (ICIIS)", pages = "443-448", year = "2020"

@onlineloremipsum, title = lstm related, url = https://www.lipsum.com,

@articleDelage, author=Panagiotis Korvesis and Stephane Besseau and Michalis Vazirgiannis, journal=2006 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'06), title=Predictive Maintenance in Aviation: Failure Prediction from Post-Flight Reports, journal = "2018 IEEE 34th International Conference on Data Engineering (ICDE)", year="2018", pages = "1414-1422"

@inproceedingsramakrishna2017design, title=Prediction of Remaining Useful Lifetime (RUL) of turbofan engine using machine learning, author=Vimala Mathew and Tom Toby and Vikram Singh and B. Maheswara Rao and M. Goutham Kumar, journal="2017 IEEE International Conference on Circuits and Systems (ICCS)", pages=306-311, year=2017

@INPROCEEDINGS9016866, author=Yan, Weili, and Jun-Hong Zhou. , title=Predictive modeling of aircraft systems failure using term frequency-inverse document frequency and random forest., journal="In 2017 IEEE International Conference on Industrial Engineering and Engineering Management (IEEM)", year=2017, pages=828-831